GDP Nowcasting App
Technical Documentation

App Designer integration, model-file changes, operating workflow, and packaging notes

Project	MATLAB App Designer GDP Nowcasting App built around the FRBNY Nowcasting codebase
Primary code areas documented	App UI/controller code plus model-side changes in load_spec.m, load_data.m, and dfm.m
Purpose of this document	Provide leadership and future maintainers with a concise technical reference for setup, use, maintenance, and handoff

1. Project summary

This application was created to provide a cleaner user interface around the FRBNY Nowcasting workflow. The app lets a user load a specification workbook and a data workbook, run the model, compare original versus updated results, visualize output on graphs, review a short summary, and inspect detailed model log output.

The App Designer layer acts as the front end. The FRBNY-based model files remain separate and handle model specification loading, data loading/transformation, and dynamic factor model estimation. This separation keeps the app easier to maintain and reduces the amount of model code embedded directly inside the UI.

2. UI structure and intended user workflow

The current app contains three main tabs.

Tab	Main components	Purpose
Main	LoadSpecFileButton, LoadDataFileButton, ExportButton, instructions text, repository hyperlink	Startup and file-loading area for the user
Graph	UIAxes, UIAxes_2, UITable3, RunGraphButton	Compares original and updated model output visually and holds the data table **(can become editable with changes)**
Summary Table	TextArea, TextArea_4, RunSummaryTableButton	Shows a short summary plus a larger detailed model log view

Typical workflow: launch the app, load the Spec workbook, load the Data workbook, optionally review or edit table values, run the graph view or summary view, and export the resulting data/output package.

3. Important app properties

The app stores the core state in private properties so that the tabs can share the same loaded data, specification, results, and logs.

Property	Role in the app
DataFilePath / SpecFilePath	Absolute paths to the currently loaded input workbooks
OriginalTable / EditedTable	Stores the original loaded table and the current working copy used for app comparison
Spec	Model specification structure returned by load_spec
OriginalResults / UpdatedResults	Dynamic factor model results structures returned by dfm
OriginalLog / UpdatedLog	Captured command-window output from model execution using evalc
Time	Time vector used for plotting
TempEditedDataFile	Temporary Excel file written from the edited table so the model can be rerun on updated data

4. App-side implementation notes

Method	Explanation
startupFcn	Adds the FRBNY/model folder tree to the MATLAB path using addpath(genpath(...)) and rehash. This solved earlier function-visibility issues for load_spec, load_data, and dfm.
loadSpecFile	Prompts for the specification workbook, stores the full path, and creates the Spec structure by calling load_spec.
loadDataFile	Prompts for the data workbook, reads the data sheet into a table, stores original and working copies, and populates UITable3.
saveEditedDataToTempFile	Converts the table from the UI back into a MATLAB table if needed and writes it to a temporary Excel workbook in the required data sheet format.
runModel	Loads the original dataset, runs dfm on the original data, loads the temporary edited workbook, reruns dfm on the updated data, and captures command-window output with evalc for both runs.
updateGraphs	Plots original and updated results separately on the two axes and protects plotting from vector-length mismatch by truncating with min(length(...)).
displayResultsInTextArea	Builds a short comparison summary in TextArea and writes the combined original/updated detailed log into TextArea_4.
exportResults	Exports app data/output to a user-selected workbook. This area should remain documented as a handoff point for any future export expansion.

5. Model-file changes to document

The main project changes were concentrated in the data-loading and integration workflow rather than the mathematical core of the model. load_data.m was reviewed to improve robustness to file selection, workbook structure, and spec/data matching. load_spec.m was updated to better handle Excel header parsing and maintain strict validation of required specification fields. dfm.m remained the core estimation engine, with its main project-facing role being to provide model results and diagnostic console output that could be used by the app for display and debugging.

5.1 load_spec.m

- Reads the specification workbook using readcell rather than depending on a narrower legacy import pattern.
- Normalizes header text by removing embedded spaces before field matching, making the file more tolerant of column-header formatting differences.
- Converts string content to character arrays before downstream processing, which helps consistency when App Designer and Excel imports return mixed text types.
- Builds the Spec structure fields required by the app and model, parses block columns, validates that all variables load on the global block, derives readable block names, and prints a model-specification summary table.

5.2 load_data.m

- Builds the cached MAT-file path more robustly with fileparts and creates the mat subfolder when it does not already exist.
- Continues to support loading from Excel while caching raw imported data to MAT for faster future reloads.
- Converts imported text values to character arrays for consistency across platforms and import modes.
- Sorts series against Spec.SeriesID and now throws explicit errors when a required SeriesID is missing or duplicated in the workbook headers. This is especially important for app debugging because it gives clear user-facing failure messages instead of vague index errors.
- Transforms the raw dataset into model-ready X, Time, and Z outputs after sorting and validation.

5.3 dfm.m

- Runs the dynamic factor model and prints informative command-window status output that can be captured by the app with evalc.
- Displays block-loading structure and factor/loading summary tables in try blocks so the app can still run even if display formatting is not available.
- Uses the supplied threshold argument or a default threshold for EM convergence control.
- Stores the smoothed output in the results structure, including both x_sm and X_sm, along with the state-space matrices and summary statistics required by later steps.
- Prints iterative EM progress information and final convergence status, which is the main detailed log content displayed on the Summary tab.

6. Known limitations / future work

- UITable3 appears prepared for editing, but a fully validated edit-tracking workflow should be treated as a later enhancement if not formally completed.
- The summary view currently uses a compact comparison metric set. A future version could report a more meaningful nowcast-specific headline value.
- Export can be expanded later to include logs, plots, and a richer results workbook instead of only the current data-oriented export behavior.

7. Handoff note

This documentation is meant to support final co-op handoff. It explains the purpose of the app, the connection between the UI and the FRBNY model files.